

LAPORAN PRAKTIKUM

Internet of Think II



Oleh :

Nama : Nabil Hasani

NPM : 25782077

Kelas : TRI (3C)

Dosen Pengampu :

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

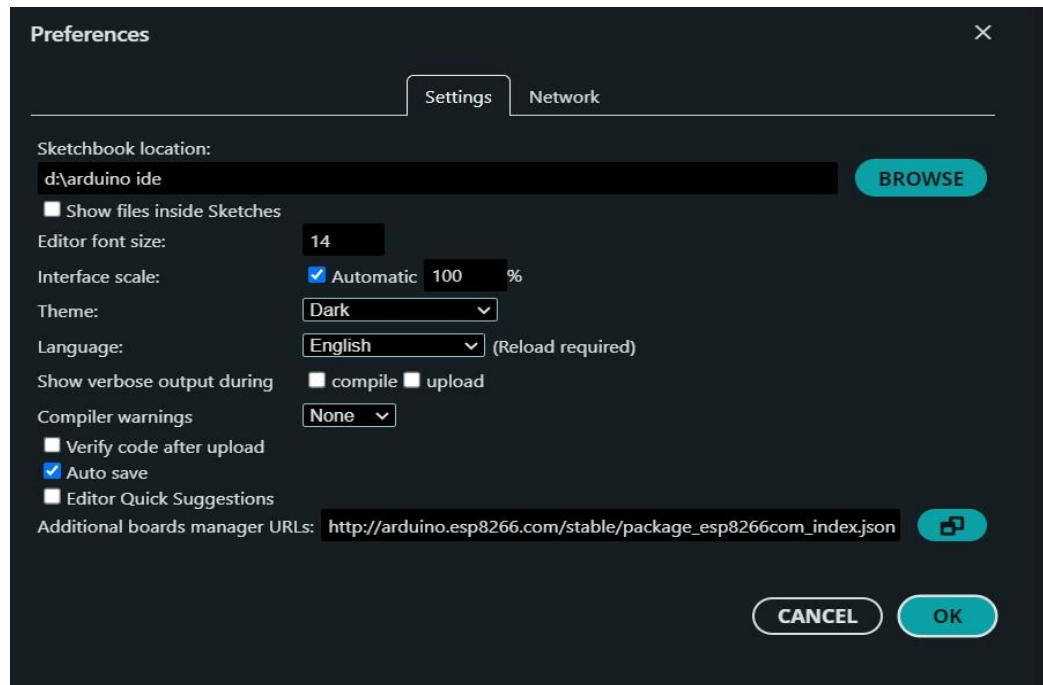
TAHUN 2026

Konfigurasi Lingkungan Pengembangan (Arduino IDE)

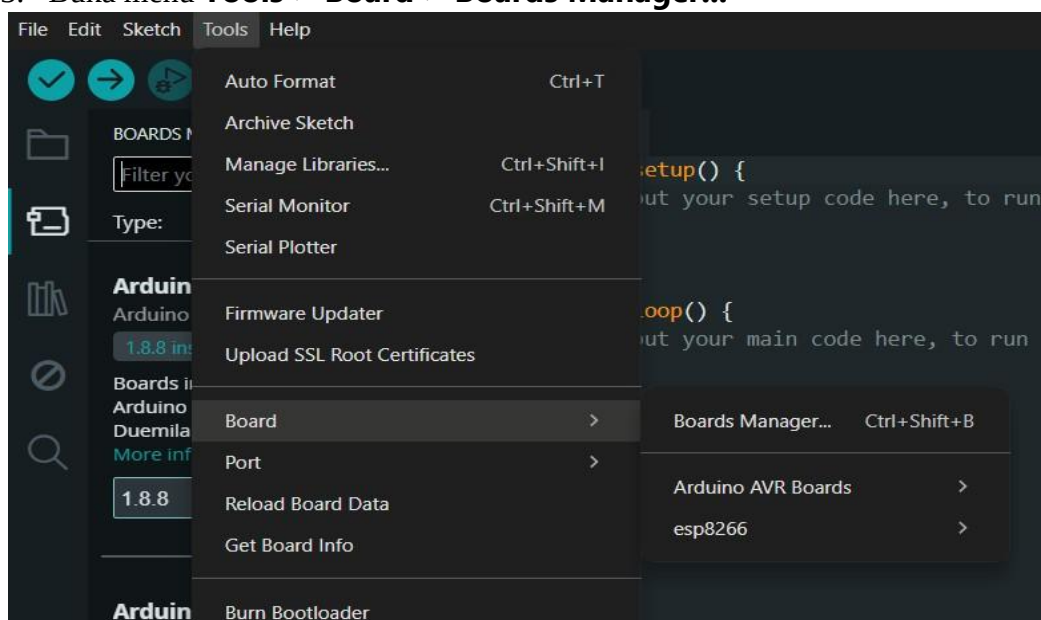
Sebelum memprogram, instal *board package* ESP8266 pada perangkat lunak Arduino IDE agar mikrokontroler dikenali.

1. Buka aplikasi **Arduino IDE**.
2. Buka menu **File > Preferences**.

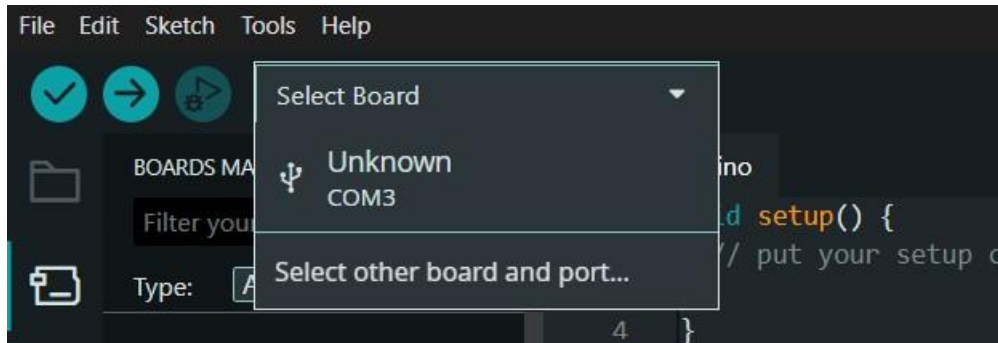
Pada kolom **Additional Boards Manager URLs**, tempelkan tautan berikut: http://arduino.esp8266.com/stable/package_esp8266com_index.json



3. Buka menu **Tools > Board > Boards Manager...**



4. Ketik "**esp8266**" di kolom pencarian, kemudian klik **Install** pada paket esp8266 by ESP8266 Community. Tunggu hingga proses unduhan selesai.
5. Konfigurasi parameter *board* Anda pada menu **Tools**:
6. **Board:** Pilih NodeMCU 1.0 (ESP-12E Module).
7. **Port:** Pilih *port COM* yang muncul saat NodeMCU dihubungkan ke laptop via USB.



Praktikum 1: Fondasi Arsitektur Internet of Things (IoT) dan Mikrokontroler

Internet of Things (IoT) merupakan suatu ekosistem yang memungkinkan perangkat fisik terhubung ke jaringan untuk mengumpulkan dan bertukar data. Mikrokontroler berfungsi sebagai salah satu bagian utama dalam sistem IoT karena dapat menerima input dari sensor maupun mengendalikan perangkat output.

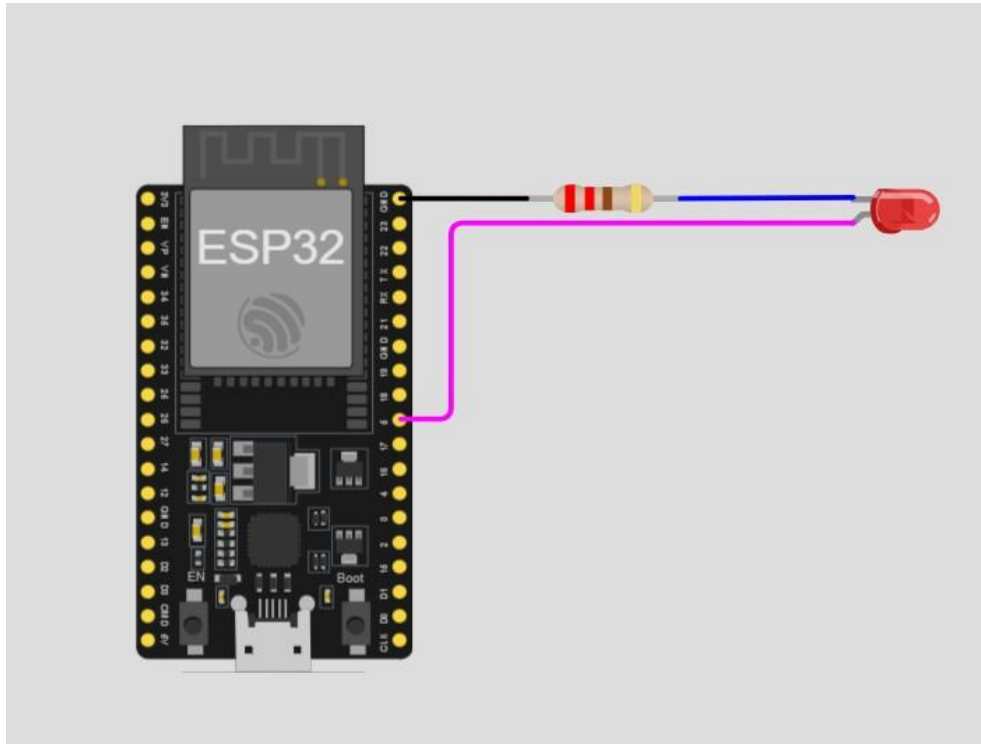
Pada praktikum ini digunakan konsep **Digital Output**, yaitu mikrokontroler memberikan sinyal digital kepada komponen seperti LED. Sinyal digital memiliki dua kondisi, yaitu **HIGH (1)** dan **LOW (0)**. Pada ESP8266, HIGH menggunakan tegangan sekitar 3,3 V sedangkan LOW berada pada 0 V atau GND.

LED digunakan sebagai indikator output. Resistor 220 Ohm digunakan sebagai pembatas arus agar arus yang melewati LED tidak terlalu besar. Modul menjelaskan bahwa LED dihubungkan dari pin D1/GPIO 5 melalui rangkaian resistor menuju GND.

Selain Digital Output, praktikum juga memperkenalkan **Digital Input** menggunakan push button. Resistor *pull-down* digunakan agar pin input tidak mengalami kondisi *floating*. Ketika tombol tidak ditekan, pin mendapatkan kondisi LOW, sedangkan ketika tombol ditekan, pin mendapatkan tegangan 3,3 V sehingga terbaca HIGH

Dokumentasi Rangkaian

Gambar 1. Rangkaian Digital Output LED



Pada gambar terlihat board mikrokontroler terhubung dengan LED menggunakan kabel dan resistor. LED digunakan sebagai indikator keluaran dari pin digital mikrokontroler

Langkah Kerja Rangkaian:

1. Pasang NodeMCU pada *breadboard*.
2. Pasang LED pada baris kosong di *breadboard*. Perhatikan kakinya:
 - Kaki yang lebih panjang adalah Anoda (Positif).
 - Kaki yang lebih pendek (dan terdapat sisi datar di tepi plastik kepala LED) adalah Katoda (Negatif).
3. Hubungkan pin D1 (GPIO 5) dari NodeMCU ke kaki Anoda LED menggunakan kabel *jumper*.
4. Hubungkan kaki Katoda LED ke salah satu ujung Resistor 220 Ohm.
5. Hubungkan ujung Resistor lainnya ke jalur GND pada NodeMCU. (*Catatan: Resistor berfungsi sebagai pembatas arus agar LED tidak terbakar*)

LED akan menyala ketika pin diberikan logika **HIGH** dan mati ketika pin diberikan logika **LOW**. Program memberikan jeda masing-masing 1000 ms sehingga LED berkedip setiap satu detik.

Langkah Kerja Pemrograman:

1. Buka Arduino IDE, klik File > New Sketch.
2. Tuliskan kode program berikut:

```
diagram.json sketch.ino Library Manager
1  const int ledPin = 5;
2
3  void setup() {
4      Serial.begin(115200);
5      pinMode(ledPin, OUTPUT);
6      Serial.println("Praktikum 1 - Digital Output Dimulai!");
7  }
8
9  void loop() {
10     digitalWrite(ledPin, HIGH);
11     Serial.println("LED Menyala");
12     delay(1000);
13
14     digitalWrite(ledPin, LOW);
15     Serial.println("LED Mati");
16     delay(1000);
17 }
```

3. Klik tombol **Upload** (ikon panah ke kanan). Tunggu hingga muncul notifikasi *Done uploading*.
4. Buka **Serial Monitor** (ikon kaca pembesar di sudut kanan atas IDE).
5. Ubah pengaturan *baud rate* pada pojok kanan bawah Serial Monitor menjadi **115200 baud**.
6. Amati hasilnya: LED fisik berkedip setiap 1 detik selaras dengan teks status yang dicetak pada Serial Monitor.

Penjelasan Singkat Kode:

- `setup()`: Blok fungsi yang dieksekusi hanya satu kali saat NodeMCU dinyalakan atau di-reset. Biasanya digunakan untuk pengaturan awal.
- `loop()`: Blok fungsi yang dieksekusi secara berulang-ulang (terus menerus) selama NodeMCU memiliki daya.
- `pinMode(ledPin, OUTPUT)`: Memberi instruksi bahwa pin D1 (GPIO 5) akan digunakan sebagai jalur keluar (*Output*) tegangan.
- `digitalWrite(ledPin, HIGH)`: Memberikan tegangan 3.3V secara digital ke pin tersebut, sehingga arus mengalir dan LED menyala.
- `delay(1000)`: Menghentikan sementara eksekusi program (jeda) selama 1000 milidetik (1 detik).

Praktikum 2: Digital Input (Membaca *Push Button*)

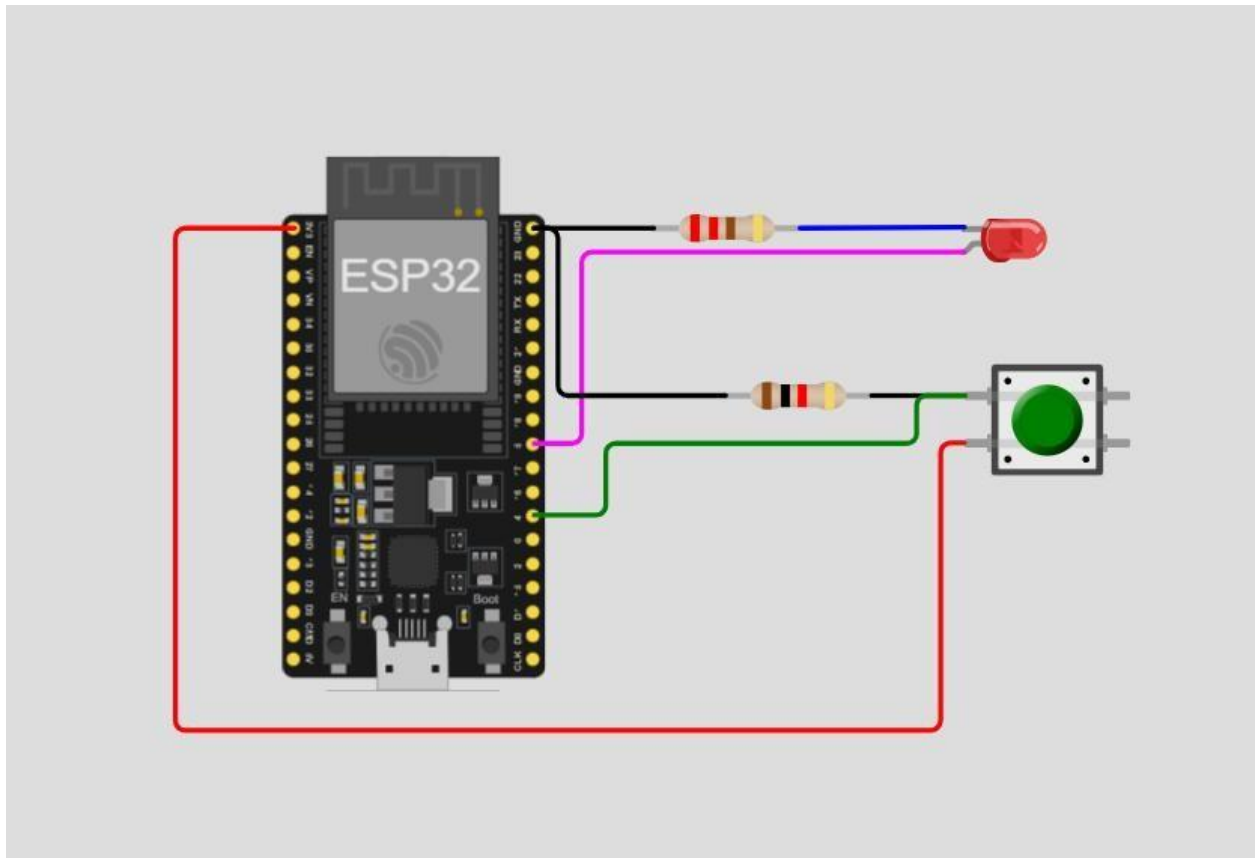
Tujuan

Praktikum 2 bertujuan untuk memahami penggunaan GPIO sebagai **Digital Input**, yaitu membaca kondisi sebuah push button dan menggunakannya untuk mengendalikan LED. Rangkaian menggunakan konsep **resistor pull-down**, sehingga ketika tombol tidak ditekan input terbaca LOW dan ketika tombol ditekan input terbaca HIGH.

Rangkaian Praktikum

Pada rangkaian yang dibuat, LED digunakan sebagai **output**, sedangkan push button digunakan sebagai **input**.

Gambar 2. Rangkaian Digital Input menggunakan Push Button



Berdasarkan rangkaian pada gambar, push button dihubungkan ke pin input melalui resistor *pull-down*. LED tetap digunakan sebagai indikator output. Pada modul, konfigurasi Praktikum 2 menggunakan **D2 (GPIO 4)** untuk push button dan **D1 (GPIO 5)** untuk LED.

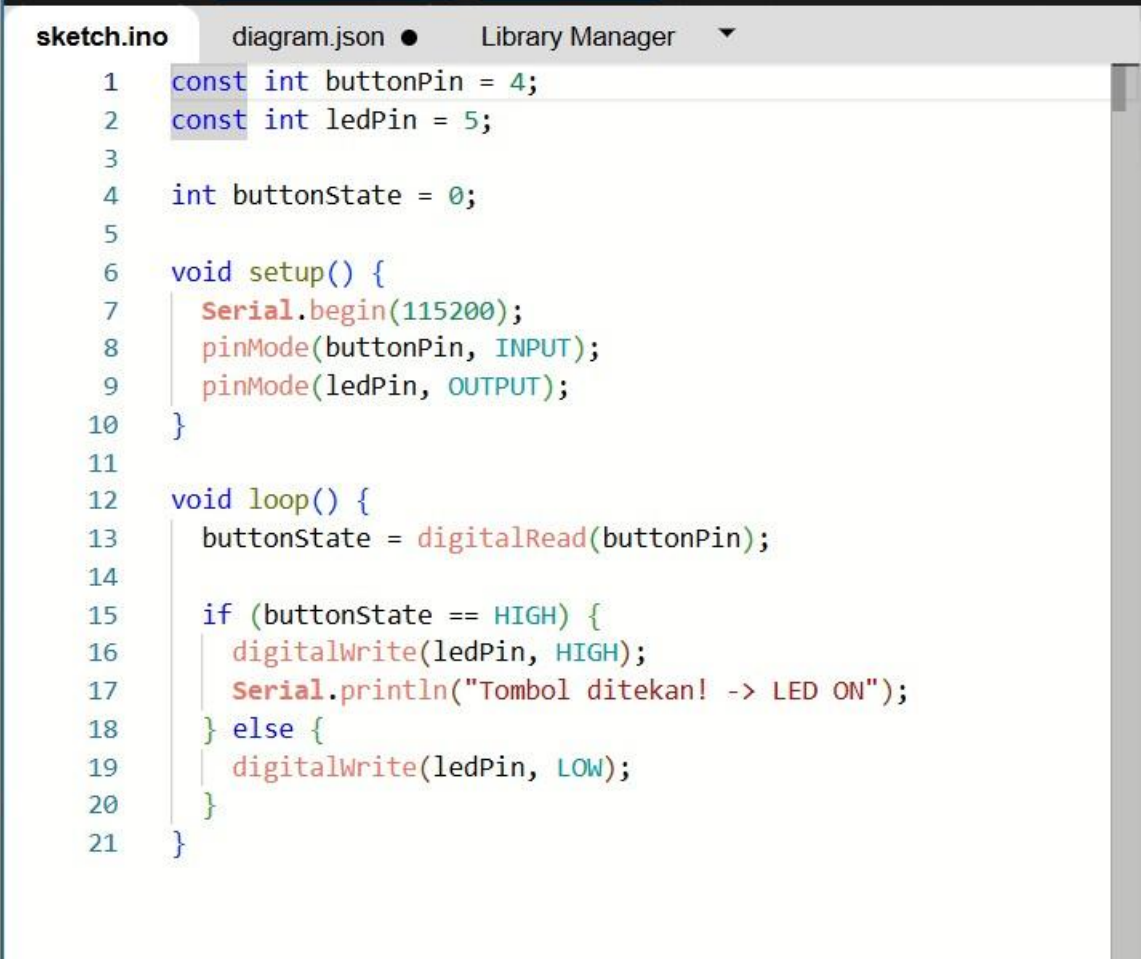
Langkah Kerja Rangkaian:

1. Biarkan rangkaian LED pada pin D1 dari Praktikum 1 tetap terpasang.
2. Pasang sebuah *push button* pada *breadboard*, melintasi garis pemisah/parit tengah.
3. Hubungkan kaki pertama *push button* ke pin 3V3 (3.3V) pada NodeMCU.
4. Hubungkan kaki kedua *push button* ke pin D2 (GPIO 4) NodeMCU.

5. Pada kaki kedua yang sama, pasang sebuah Resistor 10k Ohm, lalu hubungkan ujung lain dari resistor tersebut ke jalur GND NodeMCU.

Langkah Kerja Pemrograman:

1. Buat berkas baru (*New Sketch*) pada Arduino IDE.
2. Tuliskan kode program berikut:



```
sketch.ino  diagram.json  Library Manager
1  const int buttonPin = 4;
2  const int ledPin = 5;
3
4  int buttonState = 0;
5
6  void setup() {
7      Serial.begin(115200);
8      pinMode(buttonPin, INPUT);
9      pinMode(ledPin, OUTPUT);
10 }
11
12 void loop() {
13     buttonState = digitalRead(buttonPin);
14
15     if (buttonState == HIGH) {
16         digitalWrite(ledPin, HIGH);
17         Serial.println("Tombol ditekan! -> LED ON");
18     } else {
19         digitalWrite(ledPin, LOW);
20     }
21 }
```

3. **Upload** program ke dalam mikrokontroler.
4. Buka **Serial Monitor**.
5. Uji fungsionalitasnya: Saat tombol ditekan dan ditahan, LED akan menyala terang. Saat tombol dilepaskan, LED langsung padam.

Penjelasan Singkat Kode:

- `pinsMode(buttonPin, INPUT)`: Menetapkan pin D2 (GPIO 4) sebagai jalur masuk (*Input*) untuk mendeteksi / membaca tegangan dari luar.
- `digitalRead(buttonPin)`: Fungsi yang bertugas membaca status tegangan pada pin saat ini. Hasilnya akan berupa HIGH (ada tegangan) atau LOW (tidak ada tegangan).

- `if (buttonState == HIGH)`: Blok logika *conditional*. Karena kita menggunakan rangkaian resistor *pull-down*, menekan tombol akan mengalirkan arus 3.3V ke pin mikrokontroler. Sehingga, status terbaca HIGH dan kondisi `if` terpenuhi (LED dinyalakan).

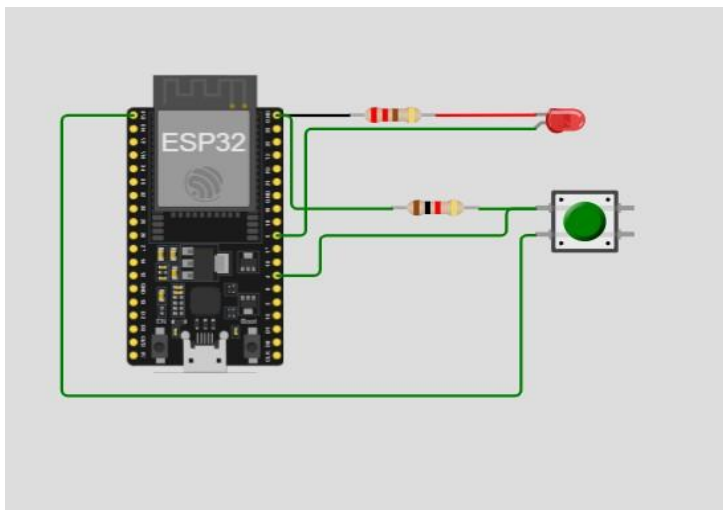
Tugas – Kendali LED dengan Tombol Push Button pada ESP32

Dasar Teori

ESP32 merupakan mikrokontroler yang memiliki pin General Purpose Input/Output (GPIO) yang dapat dikonfigurasi sebagai input maupun output. Pin yang dikonfigurasi sebagai INPUT digunakan untuk membaca sinyal digital, misalnya dari push button, yang menghasilkan nilai HIGH atau LOW. Pin yang dikonfigurasi sebagai OUTPUT digunakan untuk mengendalikan perangkat seperti LED.

Struktur kendali if-else digunakan untuk menentukan aksi program berdasarkan suatu kondisi logika. Pada praktikum ini, kondisi yang dievaluasi adalah nilai `buttonState` hasil pembacaan `digitalRead()` pada pin tombol. Dengan menukar operator perbandingan (HIGH menjadi LOW) atau menukar isi blok if dan else, perilaku keluaran program dapat dibalik tanpa mengubah rangkaian elektroniknya sama sekali, karena perubahan hanya terjadi pada level logika perangkat lunak (software).

Gambar 3. Rangkaian Sakelar Toggle



Pada rangkaian tersebut:

- LED terhubung ke **GPIO 5**.
- Push button terhubung ke **GPIO 4**.
- Push button digunakan sebagai input digital.
- LED digunakan sebagai output digital.

- Resistor digunakan untuk membatasi arus LED dan menjaga kondisi input.
- Kondisi awal LED adalah **ON**

Konfigurasi ini mengikuti instruksi tugas yang menggunakan konfigurasi hardware Praktikum 2, yaitu rangkaian *pull-down*.

Langkah Kerja Pemrograman:

1. Buat berkas baru (*New Sketch*) pada Arduino IDE.
2. Tuliskan kode program berikut:

```
const int buttonPin = 4;
const int ledPin = 5;

int buttonState = 0;

void setup() {
  Serial.begin(115200);
  pinMode(buttonPin, INPUT);
  pinMode(ledPin, OUTPUT);
}

void loop() {
  buttonState = digitalRead(buttonPin);

  if (buttonState == LOW) {
    digitalWrite(ledPin, HIGH);
    Serial.println("Tombol tidak ditekan -> LED ON");
  } else {
    digitalWrite(ledPin, LOW);
    Serial.println("Tombol ditekan -> LED OFF");
  }
}
```

3. Klik tombol **Upload** pada Arduino IDE dan tunggu hingga muncul notifikasi *Done uploading*.
4. Buka **Serial Monitor** dengan mengklik ikon kaca pembesar di sudut kanan atas Arduino IDE.
5. Atur *baud rate* Serial Monitor menjadi **115200 baud**.
6. Amati kondisi awal sistem. LED berada dalam kondisi **OFF**.
7. Tekan tombol satu kali kemudian lepaskan. LED akan berubah menjadi **ON** dan tetap menyala.
8. Tekan tombol satu kali lagi kemudian lepaskan. LED akan berubah menjadi **OFF** dan tetap mati.
9. Perhatikan **Serial Monitor**. Saat tombol pertama ditekan akan muncul pesan "**Tombol Tidak ditekan -> LED ON**", sedangkan saat tombol ditekan kembali akan muncul "**Tombol ditekan -> LED OFF**".

Perubahan yang dilakukan:

- Kondisi pembanding diubah dari `buttonState == HIGH` menjadi `buttonState == LOW`.
- Isi blok `if` dan `else` ditukar, sehingga aksi LED menyala dan mati saling bertukar posisi.
- Pesan pada Serial Monitor disesuaikan agar mencerminkan logika baru.
- Tidak ada perubahan pada deklarasi pin, konfigurasi `pinMode()`, maupun rangkaian keras.

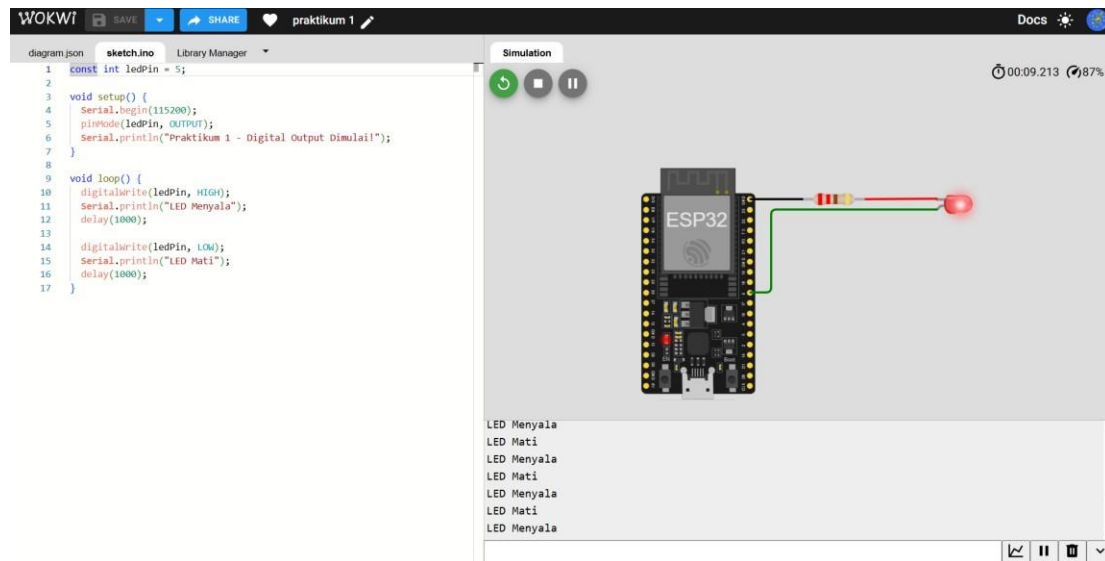
Latihan

Latihan 1 (Pemahaman Logika Pin): Berdasarkan tabel Referensi Pinout di subbab 1.3.3, jelaskan apa yang mungkin terjadi jika Anda menggunakan logika rangkaian dari Praktikum 2 (*Active-High / pull-down*), namun menyambungkan tombol tersebut ke pin **D8 (GPIO 15)**, lalu menekan tombol tersebut *tepat* pada detik di mana NodeMCU baru diberi daya?

Jika tombol pada rangkaian **Active-High / pull-down** dihubungkan ke **D8 (GPIO 15)** dan ditekan tepat saat NodeMCU baru diberi daya, maka **proses booting NodeMCU dapat gagal**. Hal ini karena GPIO 15 merupakan pin yang memiliki fungsi khusus saat *booting* dan harus berada pada kondisi tertentu. Pada modul dijelaskan bahwa **GPIO 15 (D8) ter-pull-down ke GND dan proses boot dapat gagal jika pin tersebut ditarik HIGH**.

Jadi, ketika tombol ditekan saat perangkat baru dinyalakan, tegangan **3,3 V** dapat menarik GPIO 15 menjadi **HIGH**, sehingga ESP8266 berpotensi **tidak dapat melakukan booting dengan normal**.

Latihan 2 (Implementasi Kode): Modifikasi kode pada Praktikum 1 agar LED berkedip secara *tidak simetris*, yaitu menyala sangat cepat selama 200 ms, kemudian mati agak lama selama 800 ms. Tuliskan dua baris fungsi `delay()` yang harus Anda ubah.



Latihan 3 (Analisis Sirkuit): Berdasarkan hukum aliran arus listrik searah, apa yang terjadi jika pada Praktikum 1 Anda memasang kaki LED secara terbalik (Anoda ke GND, Katoda ke D1)? Apakah program akan mengalami *error* kompilasi? Mengapa LED tidak menyala?

Jika kaki LED dipasang terbalik, yaitu **Anoda ke GND** dan **Katoda ke D1 (GPIO 5)**, maka LED **tidak akan menyala**.

Program **tidak mengalami error kompilasi**, karena pemasangan LED terbalik tidak berhubungan dengan sintaks atau kode program. Program tetap dapat di-*upload* dan dijalankan.

LED tidak menyala karena arah pemasangannya terbalik, sehingga **arus listrik tidak mengalir melalui LED dalam arah yang sesuai**. Saat GPIO 5 diberi HIGH, posisi anoda berada di GND dan katoda berada pada tegangan lebih tinggi, sehingga LED tidak mendapatkan kondisi yang diperlukan untuk menyala.

Kesimpulan: LED terbalik tidak menyebabkan error pada program, tetapi menyebabkan LED tidak menyala karena arah aliran arus pada LED tidak sesuai.

Latihan 4 (Pengembangan Algoritma): Ubah blok kondisi if-else pada Praktikum 2 sehingga perilakunya terbalik: "Saat tombol TIDAK ditekan, LED **menyala**. Saat tombol DITEKAN, LED justru **mati**."

Praktikum 1: Fondasi Arsitektur Internet of Things (IoT) dan Mikrokontroler

Internet of Things (IoT) merupakan suatu ekosistem yang memungkinkan perangkat fisik terhubung ke jaringan untuk mengumpulkan dan bertukar data. Mikrokontroler berfungsi sebagai salah satu bagian utama dalam sistem IoT karena dapat menerima input dari sensor maupun mengendalikan perangkat output.

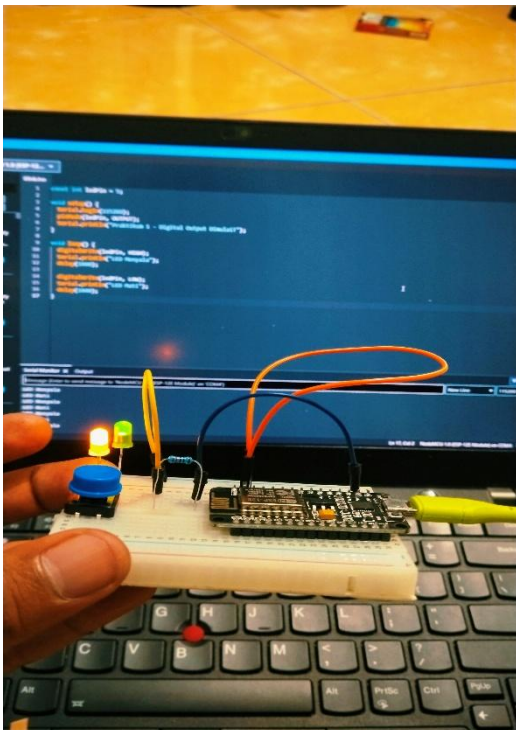
Pada praktikum ini digunakan konsep **Digital Output**, yaitu mikrokontroler memberikan sinyal digital kepada komponen seperti LED. Sinyal digital memiliki dua kondisi, yaitu **HIGH (1)** dan **LOW (0)**. Pada ESP8266, HIGH menggunakan tegangan sekitar 3,3 V sedangkan LOW berada pada 0 V atau GND.

LED digunakan sebagai indikator output. Resistor 220 Ohm digunakan sebagai pembatas arus agar arus yang melewati LED tidak terlalu besar. Modul menjelaskan bahwa LED dihubungkan dari pin D1/GPIO 5 melalui rangkaian resistor menuju GND.

Selain Digital Output, praktikum juga memperkenalkan **Digital Input** menggunakan push button. Resistor *pull-down* digunakan agar pin input tidak mengalami kondisi *floating*. Ketika tombol tidak ditekan, pin mendapatkan kondisi LOW, sedangkan ketika tombol ditekan, pin mendapatkan tegangan 3,3 V sehingga terbaca HIGH

Dokumentasi Rangkaian

Gambar 1. Rangkaian Digital Output LED



Pada gambar terlihat board mikrokontroler terhubung dengan LED menggunakan kabel dan resistor. LED digunakan sebagai indikator keluaran dari pin digital mikrokontroler

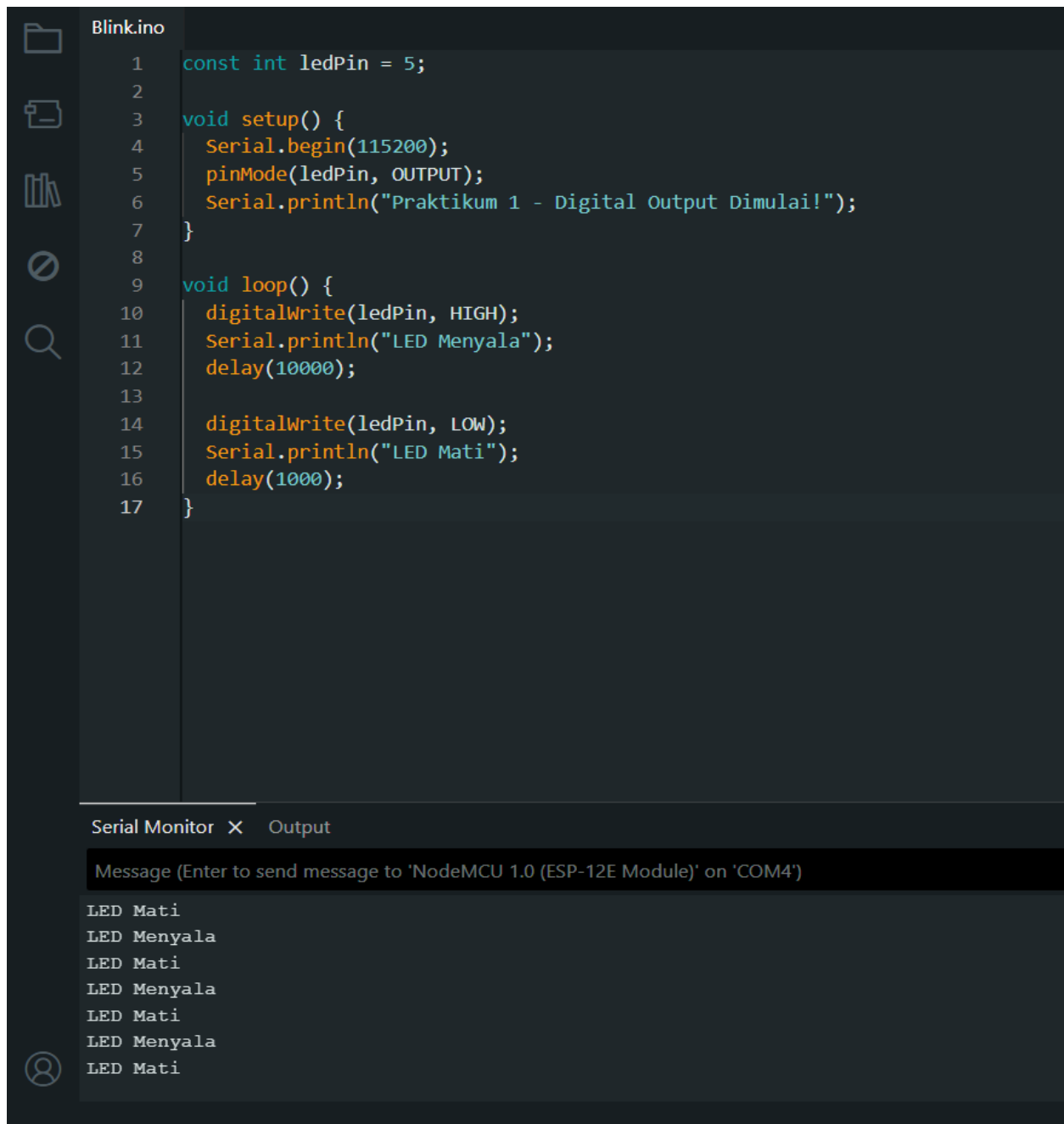
Langkah Kerja Rangkaian:

1. Pasang NodeMCU pada *breadboard*.
2. Pasang LED pada baris kosong di *breadboard*. Perhatikan kakinya:
 - Kaki yang lebih panjang adalah Anoda (Positif).
 - Kaki yang lebih pendek (dan terdapat sisi datar di tepi plastik kepala LED) adalah Katoda (Negatif).
3. Hubungkan pin D1 (GPIO 5) dari NodeMCU ke kaki Anoda LED menggunakan kabel *jumper*.
4. Hubungkan kaki Katoda LED ke salah satu ujung Resistor 220 Ohm.
5. Hubungkan ujung Resistor lainnya ke jalur GND pada NodeMCU. (*Catatan: Resistor berfungsi sebagai pembatas arus agar LED tidak terbakar*)

LED akan menyala ketika pin diberikan logika **HIGH** dan mati ketika pin diberikan logika **LOW**. Program memberikan jeda masing-masing 1000 ms sehingga LED berkedip setiap satu detik.

Langkah Kerja Pemrograman:

1. Buka Arduino IDE, klik File > New Sketch.
2. Tuliskan kode program berikut:



The image shows the Arduino IDE interface. The main window displays a file named `Blink.ino` with the following code:

```
1  const int ledPin = 5;
2
3  void setup() {
4      Serial.begin(115200);
5      pinMode(ledPin, OUTPUT);
6      Serial.println("Praktikum 1 - Digital Output Dimulai!");
7  }
8
9  void loop() {
10     digitalWrite(ledPin, HIGH);
11     Serial.println("LED Menyala");
12     delay(10000);
13
14     digitalWrite(ledPin, LOW);
15     Serial.println("LED Mati");
16     delay(1000);
17 }
```

Below the code editor, the **Serial Monitor** window is open, showing the output of the program. The output consists of alternating lines of "LED Mati" and "LED Menyala", indicating the LED is blinking. The baud rate is set to 115200.

3. Klik tombol **Upload** (ikon panah ke kanan). Tunggu hingga muncul notifikasi *Done uploading*.
4. Buka **Serial Monitor** (ikon kaca pembesar di sudut kanan atas IDE).
5. Ubah pengaturan *baud rate* pada pojok kanan bawah Serial Monitor menjadi **115200 baud**.
6. Amati hasilnya: LED fisik berkedip setiap 1 detik selaras dengan teks status yang dicetak pada Serial Monitor.

Penjelasan Singkat Kode:

- `setup()`: Blok fungsi yang dieksekusi hanya satu kali saat NodeMCU dinyalakan atau di-*reset*. Biasanya digunakan untuk pengaturan awal.
- `loop()`: Blok fungsi yang dieksekusi secara berulang-ulang (terus menerus) selama NodeMCU memiliki daya.
- `pinMode(ledPin, OUTPUT)`: Memberi instruksi bahwa pin D1 (GPIO 5) akan digunakan sebagai jalur keluar (*Output*) tegangan.
- `digitalWrite(ledPin, HIGH)`: Memberikan tegangan 3.3V secara digital ke pin tersebut, sehingga arus mengalir dan LED menyala.
- `delay(1000)`: Menghentikan sementara eksekusi program (jeda) selama 1000 milidetik (1 detik).

Praktikum 2: Digital Input (Membaca *Push Button*)

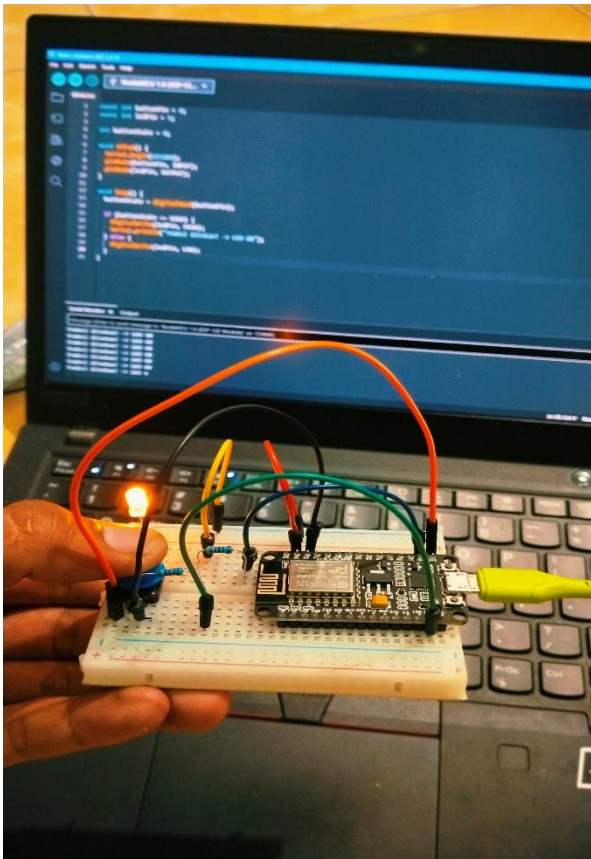
Tujuan

Praktikum 2 bertujuan untuk memahami penggunaan GPIO sebagai **Digital Input**, yaitu membaca kondisi sebuah push button dan menggunakannya untuk mengendalikan LED. Rangkaian menggunakan konsep **resistor pull-down**, sehingga ketika tombol tidak ditekan input terbaca LOW dan ketika tombol ditekan input terbaca HIGH.

Rangkaian Praktikum

Pada rangkaian yang dibuat, LED digunakan sebagai **output**, sedangkan push button digunakan sebagai **input**.

Gambar 2. Rangkaian Digital Input menggunakan Push Button



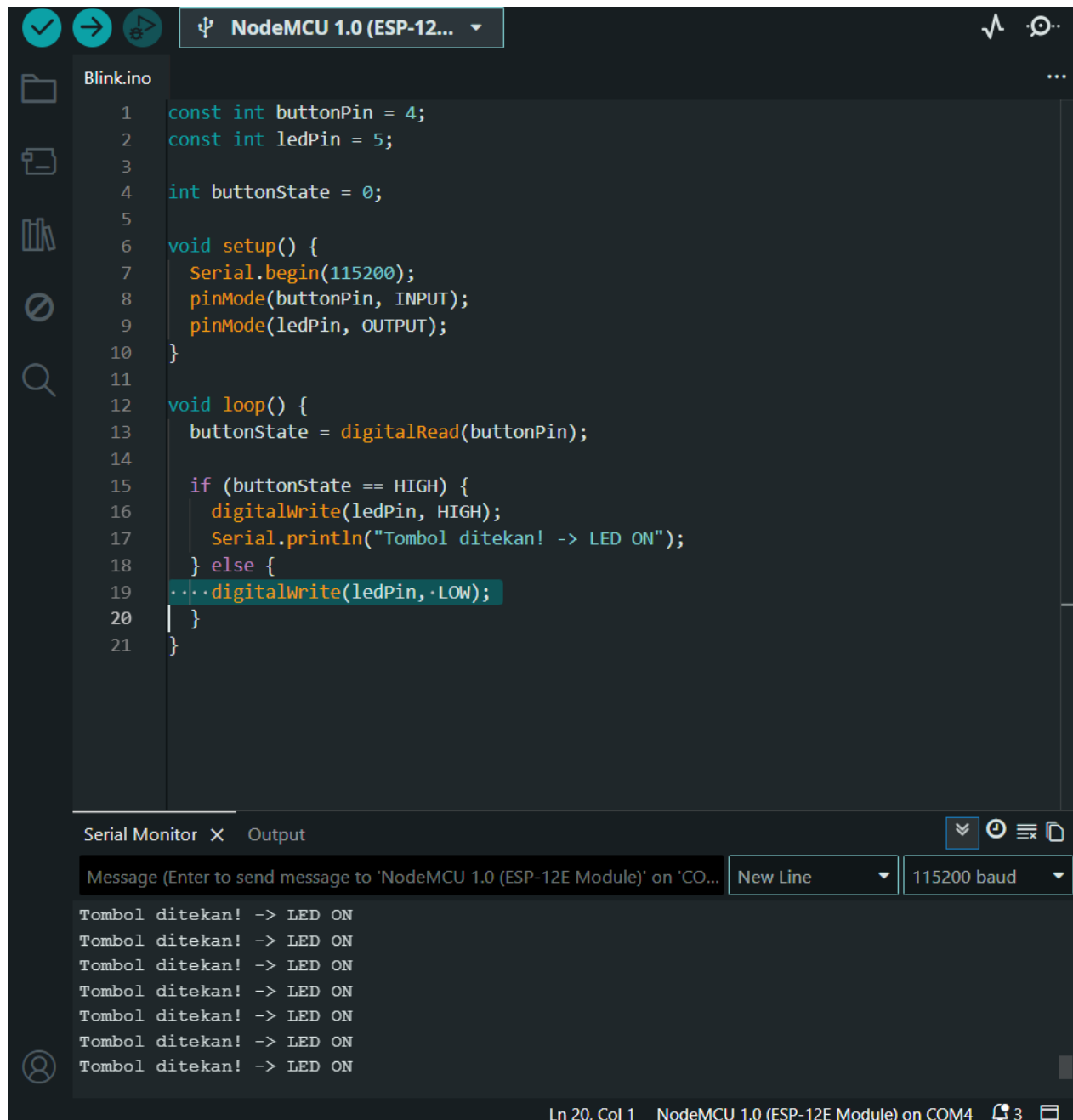
Berdasarkan rangkaian pada gambar, push button dihubungkan ke pin input melalui resistor *pull-down*. LED tetap digunakan sebagai indikator output. Pada modul, konfigurasi Praktikum 2 menggunakan **D2 (GPIO 4)** untuk push button dan **D1 (GPIO 5)** untuk LED.

Langkah Kerja Rangkaian:

1. Biarkan rangkaian LED pada pin D1 dari Praktikum 1 tetap terpasang.
2. Pasang sebuah *push button* pada *breadboard*, melintasi garis pemisah/parit tengah.
3. Hubungkan kaki pertama *push button* ke pin 3V3 (3.3V) pada NodeMCU.
4. Hubungkan kaki kedua *push button* ke pin D2 (GPIO 4) NodeMCU.
5. Pada kaki kedua yang sama, pasang sebuah Resistor 10k Ohm, lalu hubungkan ujung lain dari resistor tersebut ke jalur GND NodeMCU.

Langkah Kerja Pemrograman:

1. Buat berkas baru (*New Sketch*) pada Arduino IDE.
2. Tuliskan kode program berikut:



3. **Upload** program ke dalam mikrokontroler.
4. Buka **Serial Monitor**.
5. Uji fungsionalitasnya: Saat tombol ditekan dan ditahan, LED akan menyala terang. Saat tombol dilepaskan, LED langsung padam.

Penjelasan Singkat Kode:

- `pinMode(buttonPin, INPUT)`: Menetapkan pin D2 (GPIO 4) sebagai jalur masuk (*Input*) untuk mendeteksi / membaca tegangan dari luar.
- `digitalRead(buttonPin)`: Fungsi yang bertugas membaca status tegangan pada pin saat ini. Hasilnya akan berupa HIGH (ada tegangan) atau LOW (tidak ada tegangan).
 - `if (buttonState == HIGH)`: Blok logika *conditional*. Karena kita menggunakan rangkaian resistor *pull-down*, menekan tombol akan mengalirkan arus 3.3V ke pin mikrokontroler. Sehingga, status terbaca HIGH dan kondisi `if` terpenuhi (LED dinyalakan).

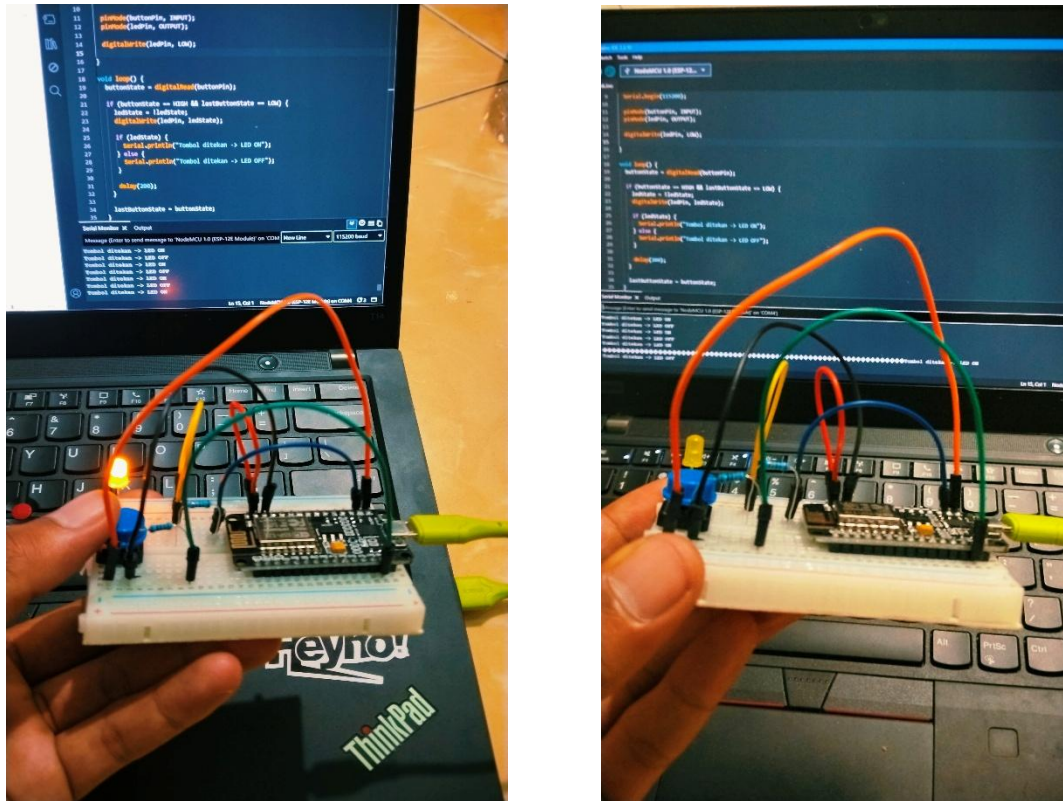
Tugas – Kendali LED dengan Tombol Push Button pada ESP32

Dasar Teori

ESP32 merupakan mikrokontroler yang memiliki pin General Purpose Input/Output (GPIO) yang dapat dikonfigurasi sebagai input maupun output. Pin yang dikonfigurasi sebagai INPUT digunakan untuk membaca sinyal digital, misalnya dari push button, yang menghasilkan nilai HIGH atau LOW. Pin yang dikonfigurasi sebagai OUTPUT digunakan untuk mengendalikan perangkat seperti LED.

Struktur kendali if-else digunakan untuk menentukan aksi program berdasarkan suatu kondisi logika. Pada praktikum ini, kondisi yang dievaluasi adalah nilai `buttonState` hasil pembacaan `digitalRead()` pada pin tombol. Dengan menukar operator perbandingan (HIGH menjadi LOW) atau menukar isi blok if dan else, perilaku keluaran program dapat dibalik tanpa mengubah rangkaian elektroniknya sama sekali, karena perubahan hanya terjadi pada level logika perangkat lunak (software).

Gambar 3. Rangkaian Sakelar Toggle



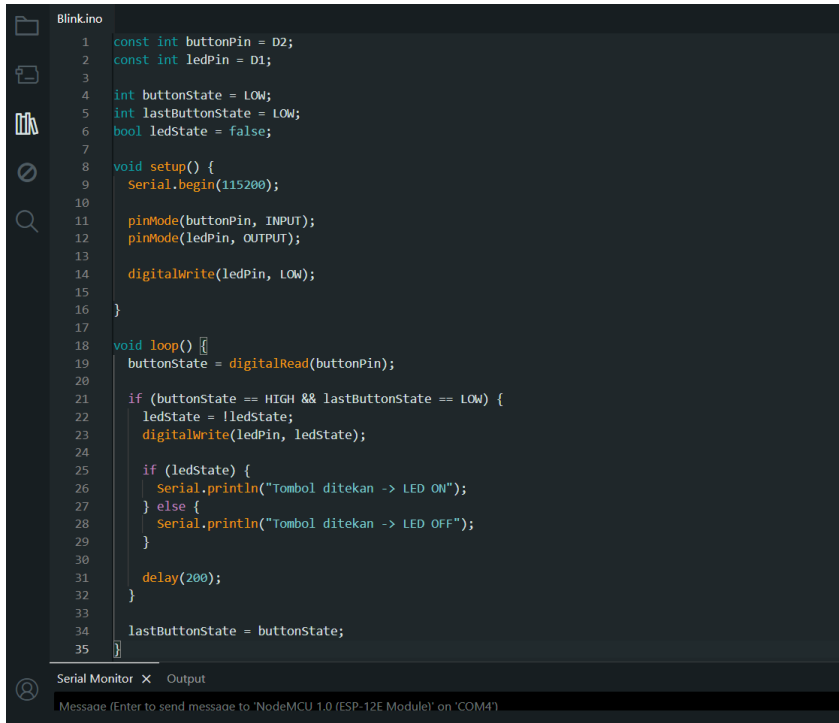
Pada rangkaian tersebut:

- LED terhubung ke **GPIO 5**.
- Push button terhubung ke **GPIO 4**.
- Push button digunakan sebagai input digital.
- LED digunakan sebagai output digital.
- Resistor digunakan untuk membatasi arus LED dan menjaga kondisi input.
- Kondisi awal LED adalah **ON**

Konfigurasi ini mengikuti instruksi tugas yang menggunakan konfigurasi hardware Praktikum 2, yaitu rangkaian *pull-down*.

Langkah Kerja Pemrograman:

1. Buat berkas baru (*New Sketch*) pada Arduino IDE.
2. Tuliskan kode program berikut:



```
1  const int buttonPin = D2;
2  const int ledPin = D1;
3
4  int buttonState = LOW;
5  int lastButtonState = LOW;
6  bool ledState = false;
7
8  void setup() {
9      Serial.begin(115200);
10
11     pinMode(buttonPin, INPUT);
12     pinMode(ledPin, OUTPUT);
13
14     digitalWrite(ledPin, LOW);
15 }
16
17 void loop() {
18     buttonState = digitalRead(buttonPin);
19
20     if (buttonState == HIGH && lastButtonState == LOW) {
21         ledState = !ledState;
22         digitalWrite(ledPin, ledState);
23
24         if (ledState) {
25             Serial.println("Tombol ditekan -> LED ON");
26         } else {
27             Serial.println("Tombol ditekan -> LED OFF");
28         }
29
30         delay(200);
31     }
32     lastButtonState = buttonState;
33 }
34
35 }
```

3. Klik tombol **Upload** pada Arduino IDE dan tunggu hingga muncul notifikasi *Done uploading*.
4. Buka **Serial Monitor** dengan mengklik ikon kaca pembesar di sudut kanan atas Arduino IDE.
5. Atur *baud rate* Serial Monitor menjadi **115200 baud**.
6. Amati kondisi awal sistem. LED berada dalam kondisi **OFF**.
7. Tekan tombol satu kali kemudian lepaskan. LED akan berubah menjadi **ON** dan tetap menyala.
8. Tekan tombol satu kali lagi kemudian lepaskan. LED akan berubah menjadi **OFF** dan tetap mati.
9. Perhatikan **Serial Monitor**. Saat tombol pertama ditekan akan muncul pesan "**Tombol Tidak ditekan -> LED ON**", sedangkan saat tombol ditekan kembali akan muncul "**Tombol ditekan -> LED OFF**".

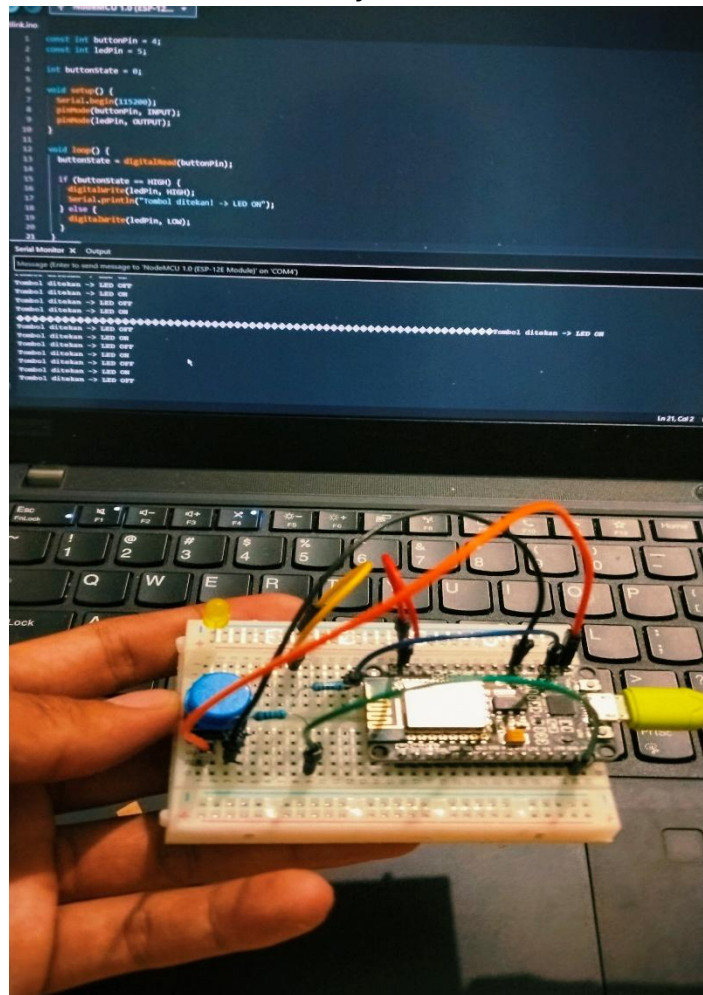
Perubahan yang dilakukan:

- Kondisi pembanding diubah dari `buttonState == HIGH` menjadi `buttonState == LOW`.
- Isi blok `if` dan `else` ditukar, sehingga aksi LED menyala dan mati saling bertukar posisi.
- Pesan pada Serial Monitor disesuaikan agar mencerminkan logika baru.

- Tidak ada perubahan pada deklarasi pin, konfigurasi `pinMode()`, maupun rangkaian keras.

Latihan

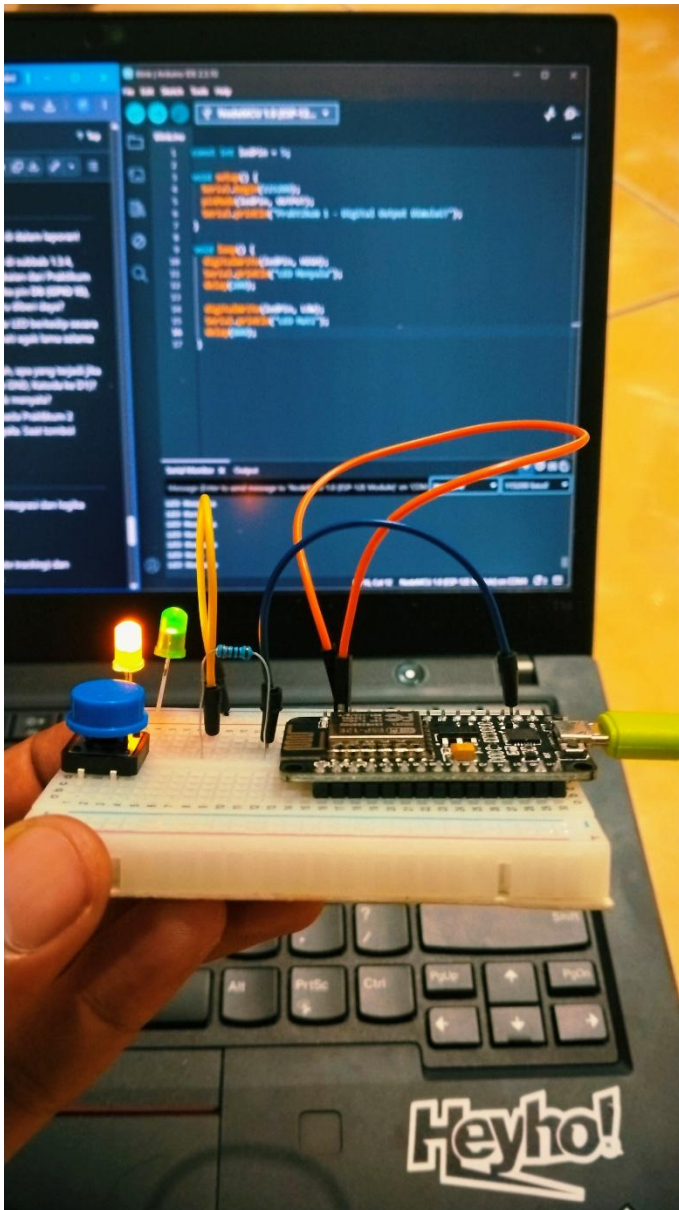
Latihan 1 (Pemahaman Logika Pin): Berdasarkan tabel Referensi Pinout di subbab 1.3.3, jelaskan apa yang mungkin terjadi jika Anda menggunakan logika rangkaian dari Praktikum 2 (Active-High / *pull-down*), namun menyambungkan tombol tersebut ke pin **D8 (GPIO 15)**, lalu menekan tombol tersebut *tepat* pada detik di mana NodeMCU baru diberi daya?



Jika tombol pada rangkaian **Active-High / pull-down** dihubungkan ke **D8 (GPIO 15)** dan ditekan tepat saat NodeMCU baru diberi daya, maka **proses booting NodeMCU dapat gagal**. Hal ini karena GPIO 15 merupakan pin yang memiliki fungsi khusus saat *booting* dan harus berada pada kondisi tertentu. Pada modul dijelaskan bahwa **GPIO 15 (D8) ter-pull-down ke GND** dan proses boot dapat gagal jika pin tersebut ditarik **HIGH**.

Jadi, ketika tombol ditekan saat perangkat baru dinyalakan, tegangan **3,3 V** dapat menarik GPIO 15 menjadi **HIGH**, sehingga ESP8266 berpotensi **tidak dapat melakukan booting dengan normal**.

Latihan 2 (Implementasi Kode): Modifikasi kode pada Praktikum 1 agar LED berkedip secara *tidak simetris*, yaitu menyala sangat cepat selama 200 ms, kemudian mati agak lama selama 800 ms. Tuliskan dua baris fungsi `delay()` yang harus Anda ubah.



```
Blinkino
1  const int ledPin = 5;
2
3  void setup() {
4      Serial.begin(115200);
5      pinMode(ledPin, OUTPUT);
6      Serial.println("Praktikum 1 - Digital Output Dimulai!");
7  }
8
9  void loop() {
10     digitalWrite(ledPin, HIGH);
11     Serial.println("LED Menyala");
12     delay(200);
13
14     digitalWrite(ledPin, LOW);
15     Serial.println("LED Mati");
16     delay(800);
17 }
```

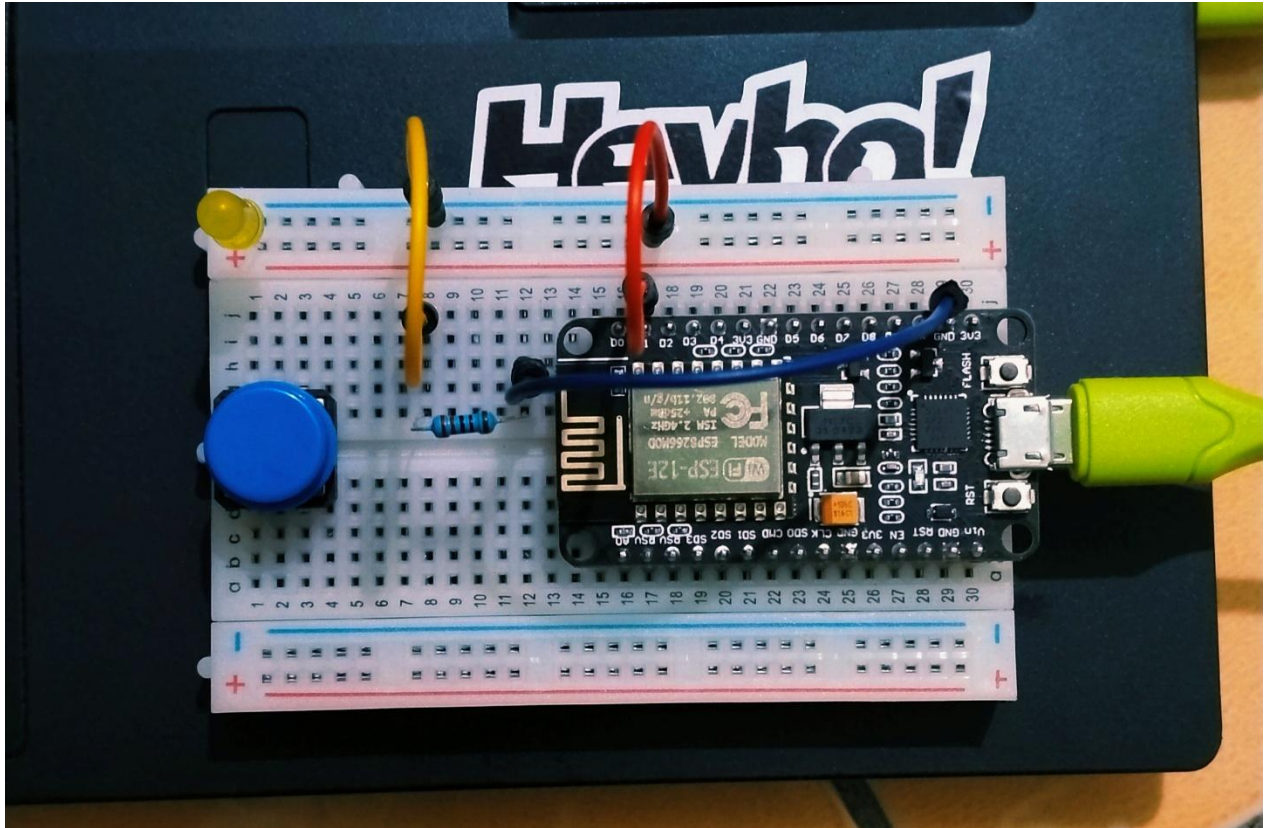
Serial Monitor X Output

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM4') New Line 115200 baud

LED Mati
LED Menyala
LED Mati
LED Menyala
LED Mati
LED Menyala
LED Mati

Ln 16, Col 12 NodeMCU 1.0 (ESP-12E Module) on COM4

Latihan 3 (Analisis Sirkuit): Berdasarkan hukum aliran arus listrik searah, apa yang terjadi jika pada Praktikum 1 Anda memasang kaki LED secara terbalik (Anoda ke GND, Katoda ke D1)? Apakah program akan mengalami *error* kompilasi? Mengapa LED tidak menyala?



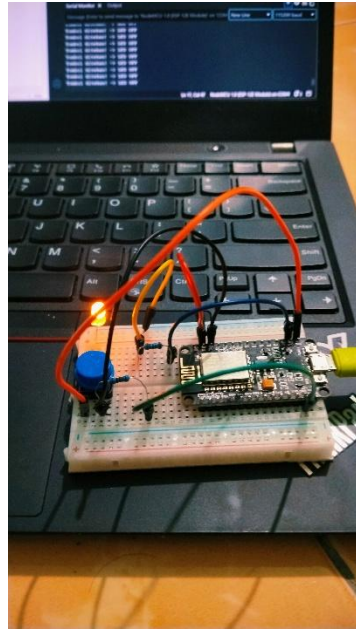
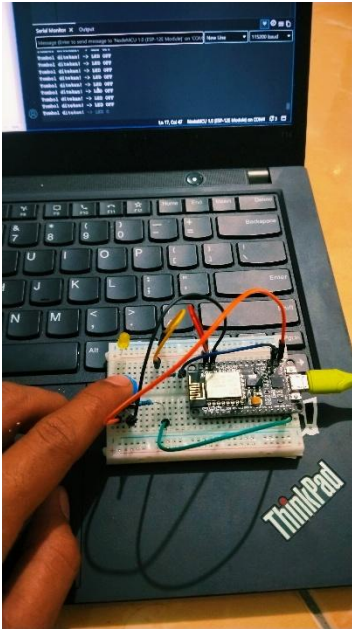
Jika kaki LED dipasang terbalik, yaitu **Anoda ke GND** dan **Katoda ke D1 (GPIO 5)**, maka LED **tidak akan menyala**.

Program **tidak mengalami error kompilasi**, karena pemasangan LED terbalik tidak berhubungan dengan sintaks atau kode program. Program tetap dapat di-*upload* dan dijalankan.

LED tidak menyala karena arah pemasangannya terbalik, sehingga **arus listrik tidak mengalir melalui LED dalam arah yang sesuai**. Saat GPIO 5 diberi HIGH, posisi anoda berada di GND dan katoda berada pada tegangan lebih tinggi, sehingga LED tidak mendapatkan kondisi yang diperlukan untuk menyala.

Kesimpulan: LED terbalik tidak menyebabkan error pada program, tetapi menyebabkan LED tidak menyala karena arah aliran arus pada LED tidak sesuai.

Latihan 4 (Pengembangan Algoritma): Ubah blok kondisi if-else pada Praktikum 2 sehingga perilakunya terbalik: "Saat tombol TIDAK ditekan, LED **menyala**. Saat tombol DITEKAN, LED justru **mati**."



```
1  const int buttonPin = 4;
2  const int ledPin = 5;
3
4  int buttonState = 0;
5
6  void setup() {
7    Serial.begin(115200);
8    pinMode(buttonPin, INPUT);
9    pinMode(ledPin, OUTPUT);
10 }
11
12 void loop() {
13   buttonState = digitalRead(buttonPin);
14
15   if (buttonState == HIGH) {
16     digitalWrite(ledPin, LOW);
17     Serial.println("Tombol ditekan! -> LED OFF");
18   } else {
19     digitalWrite(ledPin, HIGH);
20   }
21 }
```

Link Github : <https://github.com/nabilhasani2c/IoT-II.git>